Python 200
Week 3: Machine Learning: Classification

CODE THE DREAM

What is Classification?

- Week 2: regression -- predict a continuous value (price, grade)
- Week 3: classification -- predict a category

```
spam / not spam     species     fraud / legitimate
```

- Same scikit-learn API you already know:

```
create  ->  fit  ->  predict  ->  evaluate
```

- This week: KNN, Logistic Regression, and the tools to evaluate both

- Classification is everywhere:
  - spam filters, fraud detection, medical diagnosis, image recognition

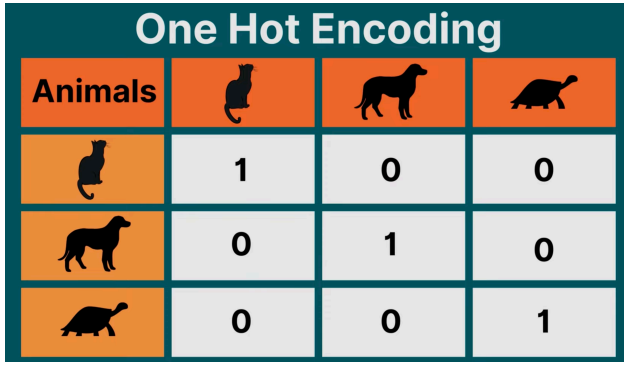
- Up next: deep learning and computer vision with PyTorch

Preprocessing: Scaling and Encoding

- Models only understand numbers
  - encode categorical strings before training
- Scaling - Normalization: MinMaxScalar.
  - Rescale features to the range [0,1]
- Scaling - Standardization: StandardScaler mean=0, std=1 (z-scores)
  - fit on train only -- prevent data leakage
- Scale-sensitive: KNN, LogReg, Neural Networks
- Scale-free: Decision Trees
- One-hot encoding: N categories -> N columns of numbers. ML model understand numbers.
  - avoids assigning a false numeric ordering

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
```

```
from sklearn.preprocessing import (
    StandardScaler, OneHotEncoder)

# Scale numeric features
scaler = StandardScaler()
X_tr_sc = scaler.fit_transform(X_train)
X_te_sc = scaler.transform(X_test)

# One-hot encode categorical column
enc = OneHotEncoder(sparse_output=False)
genres = [['jazz'],['rock'],['classical']]
encoded = enc.fit_transform(genres)
# jazz      -> [0. 1. 0.]
# rock      -> [0. 0. 1.]
# classical -> [1. 0. 0.]
```



Feature Engineering: Creating Meaningful Data

- Creates new features from existing data to capture useful information.
- Makes patterns easier for machine learning models to learn.
- Common engineering techniques include:
  - Combining features: BMI from height and weight
  - Extracting parts of a feature: Year from Timestamp
  - Binning continuous values: Age groups
  - Boolean features: is_senior based on age.

```
import pandas as pd

# Sample dataframe
df = pd.DataFrame({
    "weight_kg": [72, 68, 90],
    "height_m": [1.80, 1.70, 1.75],
    "age": [38, 25, 68]
})

# Combining features to create BMI
df["bmi"] = df["weight_kg"] / (df["height_m"] * 2)

# Binning continuous values into age groups
df["age_group"] = pd.cut(
    df["age"],
    bins=[0, 20, 30, 40, 100],
    labels=["young", "20s", "30s", "40+"]
)

# Creating a simple yes/no Boolean feature
df["is_senior"] = (df["age"] > 65).astype(int)
```

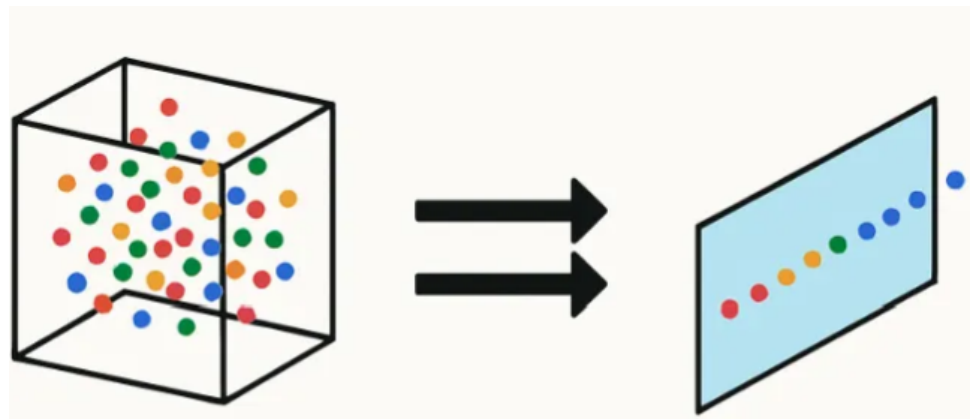
Dimensionality Reduction with PCA

- Many real datasets have correlated features
- PCA finds directions of maximum variance
  - PC1 captures the most, PC2 the next, etc.

- Demo: 1M+ pixel movie, just 2 signals
  - PC1 = room brightness (98.3% of variance)
  - PC2 = jellyfish lamp (1.5% of variance)
  - 2 numbers reconstruct near-perfectly

- Uses: visualization, noise reduction, speed

```
from sklearn.decomposition import PCA

X = frames.reshape(num_frames, -1)

pca = PCA(n_components=4, random_state=0)
pca.fit(X)

ratios = pca.explained_variance_ratio_*100
# [98.31, 1.52, 0.04, 0.02]
# PC1+PC2 = 99.8% of variance!

scores = pca.transform(X)
print(scores.shape)  # (250, 4)
```



Evaluating Classifiers: The Confusion Matrix

- A rapid Covid test is a real-life binary classifier

- 100 patients: 30 have Covid, 70 do not

		Predicted	
		Covid +	Covid -
Actual	Covid +	24 True Positive	6 False Negative
	Covid -	10 False Positive	60 True Negative

```
TP = 24  has Covid, test says positive (correct)
FN =  6  has Covid, test missed it (false negative)
FP = 10  no Covid,  test false alarm (false positive)
TN = 60  no Covid,  test says negative (correct)

Accuracy = (TP + TN) / N = (24+60)/100 = 84%
```

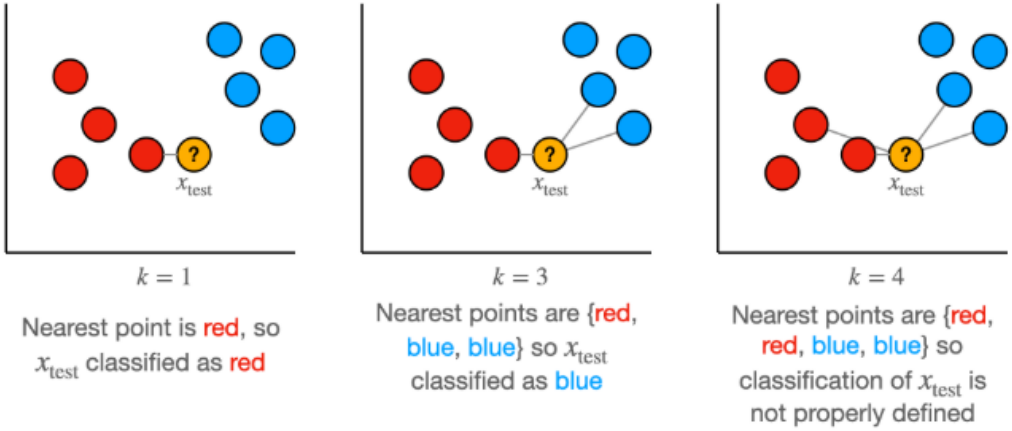
- If 2% have Covid: 'always negative' = 98% -- useless!

```
precision = TP/(TP+FP)  when it says positive, trust it?
recall    = TP/(TP+FN)  of real cases, how many caught?
F1        = 2*TP/(2*TP+FP+FN) mixture of both.
-- High if both are high.
-- Low if one is low.
-- Can't hide a weakness
```

KNN: Classify by Neighborhood

- Find the k nearest training points, take a vote
- No training phase -- stores all data, compares at predict time
- Needs feature scaling (distance-based)
- k tradeoff:
  - small k: sensitive to noise, jagged boundary
  - large k: smoother boundary, may underfit
  - tune k with cross-validation
- stratify=y: keep class proportions balanced in each split
- Iris: 4 features, 3 species, 150 flowers
  - near-perfect scores -- unusually clean data

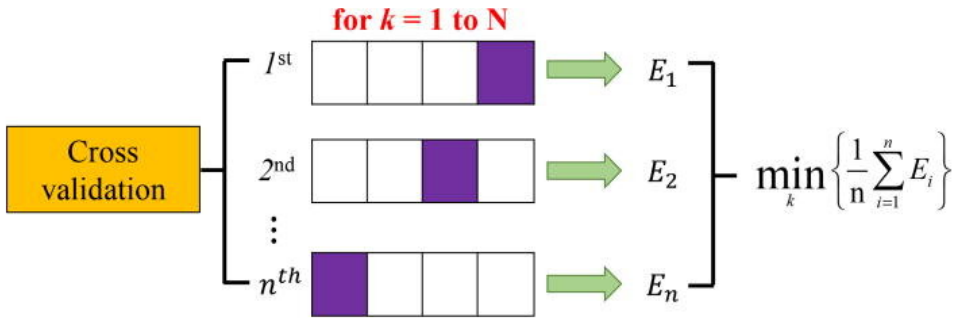
```
from sklearn.neighbors import \
    KNeighborsClassifier
from sklearn.metrics import \
    classification_report

X_tr, X_te, y_tr, y_te = train_test_split(
    X_sc, y, test_size=0.2,
    random_state=42, stratify=y)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_tr, y_tr)

preds = knn.predict(X_te)
print(classification_report(y_te, preds))
# precision  recall  f1-score
```



Cross-Validation: A More Reliable Estimate

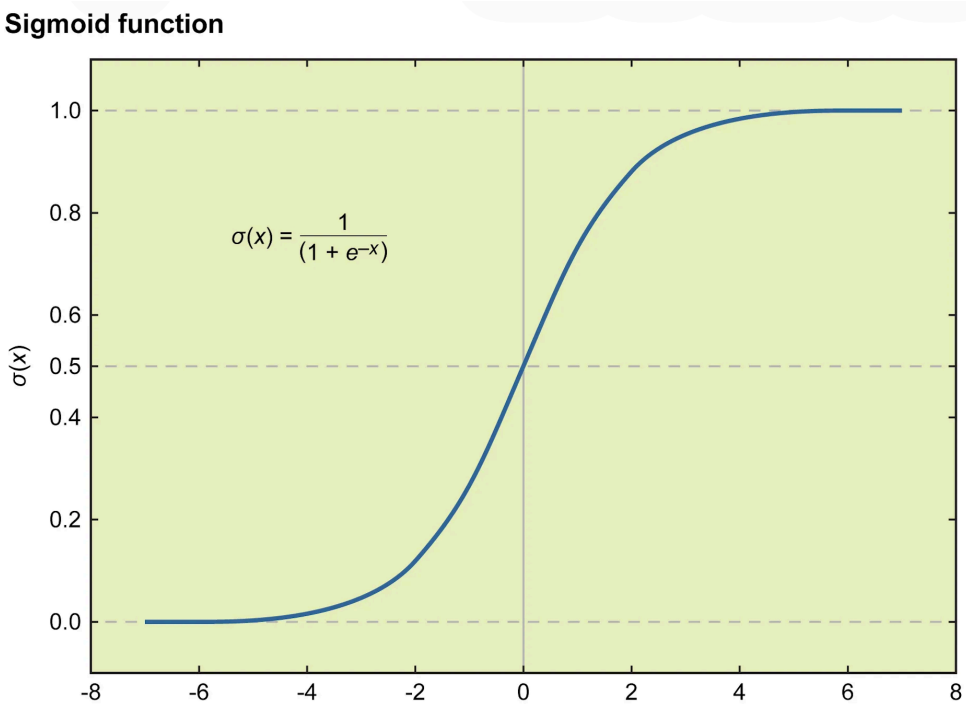
- A single train/test split can be lucky or unlucky
- k-fold: split into k folds, train on k-1, test on 1, rotate
  - Mean score: more reliable estimate of true performance
  - Std dev: how stable is the model across splits?
- Runs entirely on training data -- test set stays hidden
- Use cross-val to tune hyperparameters:
  - loop over k values, pick the one with best mean CV score

```
from sklearn.model_selection import \
    cross_val_score

knn = KNeighborsClassifier(n_neighbors=5)
scores = cross_val_score(
    knn, X_train, y_train, cv=5)
print('Scores:', scores)
# [0.97  0.93  0.97  0.90  0.93]
print('Mean:', scores.mean())  # 0.94
print('Std: ', scores.std())   # 0.027

# Tune k
for k in [1, 3, 5, 7, 9, 11]:
    knn_k = KNeighborsClassifier(k)
    s = cross_val_score(
        knn_k, X_train, y_train, cv=5)
    print(f'k={k:2d}  mean={s.mean():.3f}')
```



Logistic Regression: The Big Idea

- LR estimates a *probability*, not just a label
  - How likely is this email spam? This tumor malignant?
- A linear score z is passed through the sigmoid function
  - sigmoid squashes any number into [0, 1]
- Threshold at 0.5 -> predicted class:
  - z > 0: P > 0.5 -> class 1 | z < 0: P < 0.5 -> class 0
- Each coefficient shows one feature's influence
  - positive pushes toward class 1; negative toward class 0
  - needs scaling; fast to train; easy to interpret



Logistic Regression in Practice

- Spam dataset: 57 features (word freq, caps runs, char freq)
- Result: accuracy 0.92, macro F1 0.91
- Coefficients reveal what drives predictions:
  - positive coef -> spam signal (e.g. char_freq_$)
  - negative coef -> not spam signal (e.g. word_freq_george)
- Key advantage: every decision can be explained
  - important in production and regulated industries

```
from sklearn.linear_model import \
    LogisticRegression
import pandas as pd

lr = LogisticRegression(max_iter=1000)
lr.fit(X_train_sc, y_train)
y_pred = lr.predict(X_test_sc)
print(classification_report(y_test, y_pred))
# accuracy: 0.92   macro F1: 0.91

coef_df = pd.DataFrame({
    'feature': X.columns,
    'coef':    lr.coef_[0]
}).sort_values('coef', ascending=False)
print(coef_df.head(5).to_string(index=False))
```

Regularization: Preventing Overfitting

- Shrinks coefficients toward zero
- Prevents over-reliance on noisy features

- C controls regularization strength:
  - high C = less shrinkage (more flexible)
  - low C = more shrinkage (simpler model)

- Default C=1.0 -- tune with cross-validation

```
import numpy as np
from sklearn.linear_model import \
    LogisticRegression

for C in [0.01, 1.0, 100]:
    lr = LogisticRegression(
        C=C, max_iter=1000,
        solver='liblinear')
    lr.fit(X_train_scaled, y_train)
    total = np.abs(lr.coef_).sum()
    print(f'C={C:6}  sum={total:.2f}')

# C=0.01   sum= 0.85  (strongly shrunk)
# C=1.0    sum= 4.21  (moderate)
# C=100    sum= 8.73  (almost none)
```

Week 3 Wrap-up

- What we covered:
  - Preprocessing: scale for distance models, encode categories
  - PCA: finds correlated structure, reduces millions of dims to a few
  - Confusion matrix: TP/FP/TN/FN -- every metric derives from these
  - Precision, recall, F1: choose based on what errors cost more
  - KNN: k-nearest neighbors vote; tune k with cross-validation
  - Logistic Regression: sigmoid, probability, interpretable coefficients
  - Regularization: C controls overfitting -- tune it

- Key ideas to carry forward:
  - Always fit your scaler on training data only
  - Cross-validation > a single train/test split for hyperparameter tuning
  - Accuracy is not enough -- look at precision, recall, and F1

- Up next -- Week 4: Deep Learning and Computer Vision with PyTorch